

1. Range GCD Query with Point Updates

Code:

```
import java.util.Scanner;
public class SegmentTreeGCD {
    public static int[] tree;
    public static int n;

    public static int gcd(int a, int b) {
        if (b == 0) {
            return a;
        }
        return gcd(b, a % b);
    }

    public static void build(int[] arr, int node, int start, int end) {
        if (start == end) {
            tree[node] = arr[start];
            return;
        }
        int mid = (start + end) / 2;
        build(arr, 2 * node + 1, start, mid);
        build(arr, 2 * node + 2, mid + 1, end);
        tree[node] = gcd(tree[2 * node + 1], tree[2 * node + 2]);
    }

    public static int query(int node, int start, int end, int l, int r) {
        if (r < start || end < l || start < 0 || end >= n) {
            return 0;
        }
        if (l <= start && end <= r) {
            return tree[node];
        }
        int mid = (start + end) / 2;
        int leftGCD = query(2 * node + 1, start, mid, l, r);
        int rightGCD = query(2 * node + 2, mid + 1, end, l, r);
        return gcd(leftGCD, rightGCD);
    }

    public static void update(int node, int start, int end, int idx, int val)
    {
        if (start == end) {
            tree[node] = val;
            return;
        }
        int mid = (start + end) / 2;
        if (idx <= mid) {
            update(2 * node + 1, start, mid, idx, val);
        } else {
            update(2 * node + 2, mid + 1, end, idx, val);
        }
    }
}
```

```

        update(2 * node + 2, mid + 1, end, idx, val);
    }
    tree[node] = gcd(tree[2 * node + 1], tree[2 * node + 2]);
}

public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    System.out.print("Enter the number of elements: ");
    int a = in.nextInt();
    int[] arr = new int[a];
    for (int i = 0; i < a; i++) {
        arr[i] = in.nextInt();
    }
    n = arr.length;
    tree = new int[4 * n];
    build(arr, 0, 0, n - 1);
    int q = in.nextInt();
    for (int i = 0; i < q; i++) {
        System.out.print("Enter query type (1 for GCD query, 2 for
update): ");
        int type = in.nextInt();
        if (type == 2) {
            int l = in.nextInt();
            int r = in.nextInt();
            System.out.println(query(0, 0, n - 1, l, r));
        } else if (type == 1) {
            int idx = in.nextInt();
            int val = in.nextInt();
            update(0, 0, n - 1, idx, val);
        }
    }
    in.close();
}
}

```

2. Range Product Query with Point Updates

Code:

```

import java.util.Scanner;
public class SegmentTreeProduct {
    public static int[] tree;
    public static int n;
    public static int mod = 1000000007;

    public static void build(int[] arr, int node, int start, int end) {
        if (start == end) {
            tree[node] = arr[start];

```

```

        return;
    }
    int mid = (start + end) / 2;
    build(arr, 2 * node + 1, start, mid);
    build(arr, 2 * node + 2, mid + 1, end);
    tree[node] = (tree[2 * node + 1] * tree[2 * node + 2]) % mod;
}

public static int query(int node, int start, int end, int l, int r) {
    if (r < start || end < l || start < 0 || end >= n) {
        return 1;
    }
    if (l <= start && end <= r) {
        return tree[node];
    }
    int mid = (start + end) / 2;
    int left = query(2 * node + 1, start, mid, l, r);
    int right = query(2 * node + 2, mid + 1, end, l, r);
    return (left * right) % mod;
}

public static void update(int node, int start, int end, int idx, int val)
{
    if (start == end) {
        tree[node] = val;
        return;
    }
    int mid = (start + end) / 2;
    if (idx <= mid) {
        update(2 * node + 1, start, mid, idx, val);
    } else {
        update(2 * node + 2, mid + 1, end, idx, val);
    }
    tree[node] = tree[2 * node + 1] * tree[2 * node + 2];
}

public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    System.out.print("Enter the number of elements: ");
    int a = in.nextInt();
    int[] arr = new int[a];
    for (int i = 0; i < a; i++) {
        arr[i] = in.nextInt();
    }
    n = arr.length;
    tree = new int[4 * n];
    build(arr, 0, 0, n - 1);
    int q = in.nextInt();
    for (int i = 0; i < q; i++) {

```

```

        System.out.print("Enter query type (2 for Product query, 1 for
update): ");
        int type = in.nextInt();
        if (type == 2) {
            int l = in.nextInt();
            int r = in.nextInt();
            System.out.println(query(0, 0, n - 1, l, r));
        } else if (type == 1) {
            int idx = in.nextInt();
            int val = in.nextInt();
            update(0, 0, n - 1, idx, val);
        }
    }
    in.close();
}
}

```

3. Range Bitwise OR Query with Point Updates

Code:

```

import java.util.Scanner;
public class SegmentTreeBitwiseOr {
    public static int[] tree;
    public static int n;

    public static void build(int[] arr, int node, int start, int end) {
        if (start == end) {
            tree[node] = arr[start];
            return;
        }
        int mid = (start + end) / 2;
        build(arr, 2 * node + 1, start, mid);
        build(arr, 2 * node + 2, mid + 1, end);
        tree[node] = (tree[2 * node + 1] | tree[2 * node + 2]);
    }

    public static int query(int node, int start, int end, int l, int r) {
        if (r < start || end < l || start < 0 || end >= n) {
            return 1;
        }
        if (l <= start && end <= r) {
            return tree[node];
        }
        int mid = (start + end) / 2;
        int left = query(2 * node + 1, start, mid, l, r);
        int right = query(2 * node + 2, mid + 1, end, l, r);
        return (left | right);
    }
}

```

```

    }
    public static void update(int node, int start, int end, int idx, int val)
    {
        if (start == end) {
            tree[node] = val;
            return;
        }
        int mid = (start + end) / 2;
        if (idx <= mid) {
            update(2 * node + 1, start, mid, idx, val);
        } else {
            update(2 * node + 2, mid + 1, end, idx, val);
        }
        tree[node] = tree[2 * node + 1] | tree[2 * node + 2];
    }

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter the number of elements: ");
        int a = in.nextInt();
        int[] arr = new int[a];
        for (int i = 0; i < a; i++) {
            arr[i] = in.nextInt();
        }
        n = arr.length;
        tree = new int[4 * n];
        build(arr, 0, 0, n - 1);
        int q = in.nextInt();
        for (int i = 0; i < q; i++) {
            System.out.print("Enter query type (1 for update, 2 for Bitwise or
query): ");
            int type = in.nextInt();
            if (type == 1) {
                int idx = in.nextInt();
                int val = in.nextInt();
                update(0, 0, n - 1, idx, val);
            } else if (type == 2) {
                int l = in.nextInt();
                int r = in.nextInt();
                System.out.println(query(0, 0, n - 1, l, r));
            }
        }
        in.close();
    }
}

```

4. K-th One in a Binary Array

Code:

```
import java.util.Scanner;
public class SegmentTreekthOne {
    public static int[] tree;
    public static int n;

    public static void build(int[] arr, int node, int start, int end) {
        if (start == end) {
            tree[node] = arr[start];
            return;
        }
        int mid = (start + end) / 2;
        build(arr, 2 * node + 1, start, mid);
        build(arr, 2 * node + 2, mid + 1, end);
        tree[node] = (tree[2 * node + 1] + tree[2 * node + 2]);
    }

    public static int query(int node, int start, int end, int k) {
        if (start == end) {
            return start;
        }
        int mid = (start + end) / 2;
        int leftCount = tree[2 * node + 1];
        if (k <= leftCount)
            return query(2 * node + 1, start, mid, k);
        else
            return query(2 * node + 2, mid + 1, end, k - leftCount);
    }

    public static void update(int node, int start, int end, int idx, int val)
    {
        if (start == end) {
            tree[node] = val;
            return;
        }
        int mid = (start + end) / 2;
        if (idx <= mid) {
            update(2 * node + 1, start, mid, idx, val);
        } else {
            update(2 * node + 2, mid + 1, end, idx, val);
        }
        tree[node] = tree[2 * node + 1] + tree[2 * node + 2];
    }

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter the number of elements: ");
        int a = in.nextInt();
    }
}
```

```

int[] arr = new int[a];
for (int i = 0; i < a; i++) {
    arr[i] = in.nextInt();
}
n = arr.length;
tree = new int[4 * n];
build(arr, 0, 0, n - 1);
int q = in.nextInt();
for (int i = 0; i < q; i++) {
    System.out.print("Enter query type (1 for update,2 for Bitwise or
query): ");
    int type = in.nextInt();
    if (type == 1) {
        int idx = in.nextInt();
        int val = in.nextInt();
        update(0, 0, n - 1, idx, val);
    } else if (type == 2) {
        int k = in.nextInt();
        if (tree[0] < k)
            System.out.println(-1);
        else
            System.out.println(query(0, 0, n - 1, k));
    }
}
in.close();
}
}

```

5. Count Zeros in a Range with Point Updates

Code:

```

import java.util.Scanner;

public class SegmentTreeCountZeros {
    static class Node{
        int count;
    }
    public static Node[] tree;
    public static int n;

    public static void build(int[] arr, int node, int start, int end) {
        if (start == end) {
            if(arr[start]==0)
                tree[node].count = 1;
            else
                tree[node].count = 0;
            return;
        }
    }
}

```

```

    }
    int mid = (start + end) / 2;
    build(arr, 2 * node + 1, start, mid);
    build(arr, 2 * node + 2, mid + 1, end);
    tree[node].count = tree[2 * node + 1].count + tree[2 * node + 2].count;
}

public static int query(int node, int start, int end, int l, int r) {
    if(start > r || end < l)
        return 0;
    if(l <= start && end <= r)
        return tree[node].count;
    int mid = (start + end) / 2;
    int left = query(2 * node + 1, start, mid, l, r);
    int right = query(2 * node + 2, mid + 1, end, l, r);
    return left + right;
}

public static void update(int node, int start, int end, int idx, int val)
{
    if (start == end) {
        if(val == 0)
            tree[node].count = 1;
        else
            tree[node].count = 0;
        return;
    }
    int mid = (start + end) / 2;
    if (idx <= mid) {
        update(2 * node + 1, start, mid, idx, val);
    } else {
        update(2 * node + 2, mid + 1, end, idx, val);
    }
    tree[node].count = tree[2 * node + 1].count + tree[2 * node +
2].count;
}

public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    System.out.print("Enter the number of elements: ");
    int a = in.nextInt();
    int[] arr = new int[a];
    for (int i = 0; i < a; i++) {
        arr[i] = in.nextInt();
    }
    n = arr.length;
    tree = new Node[4 * n];
    for (int i = 0; i < 4 * n; i++)
        tree[i] = new Node();
}

```



```

        build(arr, 0, 0, n - 1);
        int q = in.nextInt();
        for (int i = 0; i < q; i++) {
            System.out.print("Enter query type (1 for update,2 for Bitwise or
query): ");
            int type = in.nextInt();
            if (type == 1) {
                int idx = in.nextInt();
                int val = in.nextInt();
                update(0, 0, n - 1, idx, val);
            } else if (type == 2) {
                int l = in.nextInt();
                int r = in.nextInt();
                System.out.println(query(0, 0, n - 1, l,r));
            }
        }
        in.close();
    }
}

```